

Projet Back-End – EFREI Paris

Nom du projet : orium_agence_scraper

Binôme : Simon Sacha Roux & Dorian

Lien GitHub : https://github.com/Drx93/orium_agence_scrapper

Expression de besoin :

Ce projet a pour objectif de concevoir une API REST complète permettant de centraliser et exposer des informations sur les restaurants (nom, adresse, géolocalisation, rating, type, etc.) issues d'un scraping.

Objectifs : Fournir un back-end robuste, sécurisé et documenté. Ainsi qu'une interface front end optimale pour l'affichage des données de l'API et un script de scraping pour récupérer des données sur les restaurants.

Rôles : Admin, User.

Scénarios utilisateurs :

- User signup/login
- Recherche d'un restaurant
- Consultation d'une fiche restaurant
- Ajout dans la catégorie contacté et/ou consulté et/ou favori
- Liste des contactés/consultés/favoris
- En tant qu'admin, le CRUD complet

Cas d'usages clés de l'API : Dans le cadre d'Orium Agence, société de référencement des commerces/établissements/entreprises, nous avons besoin de pouvoir suivre l'avancement des démarches que nous faisons auprès de nos clients ainsi que de trouver de nouveaux clients plus simplement.

En listant tous les restaurants nous allons pouvoir nous garantir une base de données très large, nous donnant une avance sur nos concurrents et nous permettant de scale la prospection.

Cet outil n'est que la première version, il se verra apporter de nouvelles fonctionnalités afin d'étoffer les données des restaurants / restaurateurs. Nous pourrions même envisager de le vendre à nos concurrents ou à des agences qui cherches à se procurer un certain nombre de données.

Analyse des sécurités :

CORS	Autorise les origines spécifique (frontend) en l'occurrence sur le projet pour le moment nous acceptons toutes les url. Mais l'implémentation d'origines spécifiques est présente
Rate Limiting	Limite les requêtes par IP à 1000 par 5min
Token JWT	Acces token + Refresh + rôle
Rôle et permissions	Admin -> toutes User -> lecture
Hashing	Bcrypt pour les mots de passes
Gestion des erreurs	Nous avons des fichiers de tests pour les requêtes sur les users et des requêtes sur les restaurants. Sinon nous avons mis en place des console log afin de cibler les problèmes durant le dev. Ainsi que des try/except et des retour sur les requêtes (retour 200 etc)

Choix des technologies :

Node.js

Node.js constitue la fondation du back-end. Il permet d'exécuter du JavaScript côté serveur pour la création d'APIs asynchrones.

- **Rôle** : exécuter le serveur applicatif et gérer les requêtes HTTP entrantes.
- **Pourquoi ce choix** : un langage unique (JavaScript) entre le front et le back simplifie la maintenance.
- **Avantages** : rapidité d'exécution, pleins modules via npm.
- **Limites** : moins adapté aux traitements lourds CPU (mais pas notre cas).
- **Intégration** : utilisé comme environnement d'exécution principal via node et gestion des dépendances avec npm.

Express.js

Express est le framework principal du projet. Léger, il offre une base solide pour construire notre architecture REST.

- **Rôle** : définir les routes, centraliser les middlewares et structurer le serveur selon le modèle MVC.
- **Pourquoi ce choix** : sa simplicité et sa rapidité.
- **Avantages** : simplicité d'implémentation, compatibilité avec notre projet.
- **Limites** : /
- **Intégration** : routes organisées par module (/auth, /restaurants, /users), avec middlewares tels que cors et express-rate-limit.

PostgreSQL

Le choix de PostgreSQL a été logique car nous l'avons utilisé auparavant dans d'autres projets d'API.

- **Rôle** : assurer la persistance des données structurées et les relations entre les entités (notamment les jointures entre utilisateurs et restaurants).
- **Pourquoi ce choix** : car nous sommes très familiarisés avec cette techno
- **Avantages** : intégrité des données, gestion fine des contraintes.
- **Limites** : structure de schéma plus rigide qu'une base NoSQL, ce qui demande une planification initiale rigoureuse.
- **Intégration** : connexion via le module pg, tables créées selon le MLD du projet (roles, users, user_restaurant_status).

MongoDB

MongoDB est utilisée en complément pour stocker les données non relationnelles issues du scraping, comme les informations de restaurants.

Ce type de donnée, souvent hétérogène et évolutive, se prête particulièrement bien au format document JSON qu'on extrait avec le scraper.

- **Rôle** : héberger la collection restaurants, avec des champs dynamiques (emails multiples, options de service, etc.).
- **Pourquoi ce choix** : grande flexibilité de schéma et performances élevées pour la lecture/écriture de gros volumes de restaurants.
- **Avantages** : rapidité, indexation géospatiale native (latitude/longitude).
- **Limites** : cohérence référentielle limitée, logique applicative nécessaire pour certaines validations.
- **Intégration** : utilisation de mongoose pour définir des schémas et valider les documents avant insertion.

JWT (JSON Web Token)

On adopte une authentification JWT.

- **Rôle** : authentifier les utilisateurs et gérer les droits d'accès selon leur rôle.
- **Pourquoi ce choix** : nous étions déjà familiarisés avec JWT lors de précédents projets.
- **Avantages** : facilement intégrable dans Express.
- **Limites** : la révocation de jetons.
- **Intégration** : utilisation du module jsonwebtoken avec une clé secrète stockée dans les variables d'environnement.

Bcrypt

Afin de garantir la sécurité des mots de passe, le hashage est fait à l'aide de la librairie bcrypt.

- **Rôle** : hacher les mots de passe lors de l'inscription et les comparer lors de la connexion.
- **Pourquoi ce choix** : Bcrypt applique une complexité configurable.
- **Avantages** : implémentation simple.
- **Limites** : /
- **Intégration** : appelé au moment de la création d'un utilisateur et lors de la vérification des identifiants.

CORS

La configuration CORS est essentielle pour autoriser uniquement le frontend du projet à communiquer avec l'API.

- **Rôle** : restreindre l'accès à certaines origines pour éviter les appels non autorisés.
- **Pourquoi ce choix** : garantir que seules les requêtes provenant du domaine frontend (<http://localhost:3000>) soient acceptées.
- **Avantages** : sécurité renforcée côté navigateur, empêche les attaques inter-origines.
- **Limites** : /

- **Intégration** : middleware cors avec une liste d'origines autorisées définie dans les variables d'environnement.

Rate Limiter

Pour limiter les risques d'abus ou d'attaques, un système de limitation du nombre de requêtes est mis en place.

- **Rôle** : contrôler la fréquence des requêtes d'une même adresse IP.
- **Pourquoi ce choix** : prévenir le brute-force sur les endpoints sensibles (/auth/login) et protéger les ressources.
- **Avantages** : facile à configurer, améliore la sécurité globale.
- **Limites** : /
- **Intégration** : module express-rate-limit, avec une fenêtre de 5 minutes et 100 requêtes maximum par IP.

Swagger

Swagger est utilisé pour la documentation technique de l'API.

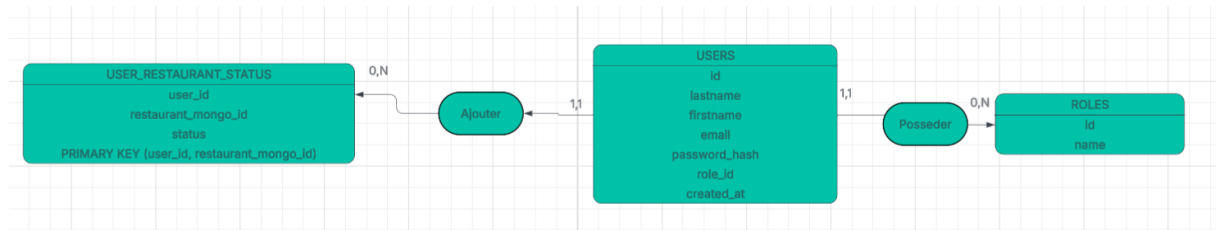
Il permet de visualiser les endpoints, les schémas de données et de tester les requêtes directement depuis une interface web.

- **Rôle** : générer une documentation claire et interactive pour les développeurs.
- **Pourquoi ce choix** : Pratique d'utilisation.
- **Avantages** : gain de temps pour les tests, transparence du fonctionnement de l'API.
- **Limites** : très long à mettre en place.
- **Intégration** : route /api-docs exposant la documentation générée avec swagger-jsdoc et swagger-ui-express.

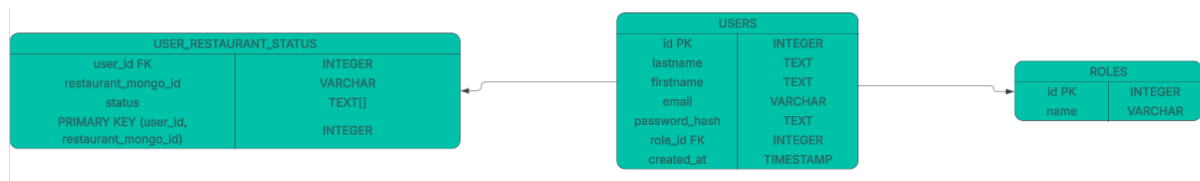
- ▼ src
 - ▼ controllers
 - JS auth.controller.js
 - JS restaurants.controller.js
 - JS userRestaurants.controller.js
 - JS users.controller.js
 - > db
 - ▼ middlewares
 - JS auth.js
 - JS rateLimiter.js
 - JS requireRole.js
 - ▼ models
 - JS refreshToken.model.js
 - JS restaurants.model.js
 - JS userRestaurants.model.js
 - JS users.model.js
 - > routes
 - > utils
 - JS app.js
 - JS server.js
- ▼ structure-bdd
 - bdd_sql.sql
- ▼ tests
 - JS auth.test.js
 - JS restaurants.test.js
- ⚙ .env
- ⚙ .env.example
- 🔒 .gitignore
- { } package-lock.json
- { } package.json

MCD et MLD :

MCD Merise :



MLD :



Les données qui nécessitent une intégrité sont stockées dans PostgreSQL et le reste dans Mongo pour une plus grande liberté de la forme de la base de données.

Explications de code :

Route :

```
const ctrl = require("../controllers/users.controller");

router.get("/", ctrl.listUsers);
router.get("/:id", ctrl.getUser);
router.post("/", ctrl.createUser);
router.put("/:id", ctrl.updateUser);
router.delete("/:id", ctrl.deleteUser);

module.exports = router;
```

Voici les routes du fichier users.routes.js, sur ce fichier nous appelons les différentes fonctions du users.controller en fonction des url et du type de requête qui est effectué

Controller :

```
const User = require('../models/users.model');

exports.listUsers = async (req, res, next) => {
  try {
    const { q, limit = 50, offset = 0 } = req.query;
    const data = await User.findAll({ q, limit: Number(limit), offset: Number(offset) });
    return res.status(200).json({ total: data.length, data });
  } catch (e) { next(e); }
};
```

Pour continuer avec les fichiers users, nous avons ici la fonction pour lister tous les utilisateurs qui sont présents dans la base de données PostgreSQL avec un nombre maximal de 50 users qui sont retournés

Model PostgreSQL :

```

const { Pool } = require('pg');
const dotenv = require('dotenv');

dotenv.config();

const connectionString = process.env.DATABASE_URL;

if (!connectionString) {
  console.warn('DATABASE_URL not set in environment; Postgres pool will not be created.');
}

const pool = connectionString ? new Pool({ connectionString }) : null;

async function query(text, params) {
  if (!pool) throw new Error('Postgres pool not initialized (DATABASE_URL missing)');
  const start = Date.now();
  const res = await pool.query(text, params);
  const duration = Date.now() - start;
  // console.debug('executed query', { text, duration, rows: res.rowCount });
  return res;
}

```

Pour le model PostgreSQL on fait appel à la Pool pour gérer la connexion et la déconnexion. Pour la connexion on a créé une fonction asynchrone (même principe que await avec mongoose).

Model Mongo :

```

import mongoose from "mongoose";
import { MONGO_URI } from "../env.js"; // Assurez-vous que MONGO_URI est défini dans env.js

export const connectToMongoDB = async () => {
  try {
    await mongoose.connect(MONGO_URI);
    console.log("Connecté à MongoDB");

    mongoose.connection.on("error", (err) =>
      console.error("Erreur de connexion MongoDB :", err)
    );
    mongoose.connection.on("disconnected", () =>
      console.log("MongoDB déconnecté")
    );

    // Gestion de la fermeture propre de la connexion
    const gracefulShutdown = async () => {
      await mongoose.disconnect();
      console.log("Connexion MongoDB fermée");
      process.exit(0);
    };
    process.on("SIGINT", gracefulShutdown);
    process.on("SIGTERM", gracefulShutdown);
  } catch (error) {
    console.error("Erreur de connexion à MongoDB :", error);
    throw error;
  }
};

```

Voici le model de mongo, to d'abord on lance une connexion à la base de données et en fonction des retours que fait mongoose, on affiche s'il y a une erreur ou si on se déconnecte de la base. Par ailleurs pour fermer la connexion on utilise await le temps d'attendre que tout se fait proprement avant d'afficher des logs

Middleware JWT :

```

const jwt = require('jsonwebtoken');

/**
 * Middleware d'authentification JWT.
 * - Lit l'en-tête Authorization: Bearer <token>
 * - Vérifie la signature et l'expiration via process.env.JWT_SECRET
 * - Injecte req.user = { id, role } si token valide
 * - Renvoie 401 si token manquant ou invalide
 */
module.exports = function auth(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;

  if (!token) return res.status(401).json({ error: 'Token manquant' });

  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    // payload should contain at least { sub, role }
    req.user = { id: payload.sub, role: payload.role };
    return next();
  } catch (err) {
    return res.status(401).json({ error: 'Token invalide ou expiré' });
  }
};

```

Pour le middleware JWT on crée une autorisation si le Bearer token est valide s'il ne l'est pas on renvoie un status 401

Rate Limiter :

```

1  const rateLimit = require('express-rate-limit');
2
3  /**
4   * Exporte une instance de rate limiter par défaut et une factory
5   * pour créer des limiteurs personnalisés.
6   */
7  const defaultLimiter = rateLimit({
8    windowMs: 5 * 60 * 1000, // 5 minutes
9    max: 1000, // limite chaque IP à 1000 requêtes par windowMs
10   standardHeaders: true,
11   legacyHeaders: false,
12   message: { error: 'Trop de requêtes, réessayez plus tard.' }
13 });
14

```

Dans ce rate limiter on a décidé de mettre un maximum de 1000 requêtes par 5 minutes par IP. Si on atteint la limite alors tout accès est bloqué et on a le message « Trop de requêtes, réessayez plus tard »

Swagger :

```
/**  
 * Modèle User basé sur Postgres.  
 * Colonnes : id (int), lastname, firstname, email, password_hash, role_id, created_at  
 */
```

Ici on explicite les différentes colonnes de notre modèle User

Données utilisées :

Pour ce projet nous avons utilisé des données récupérées sur une api payante (version gratuite) puis nous avons appliqué des regex pour avoir les accents plutôt que des caractères spéciaux et splitter certaines chaines de textes afin d'avoir une liste d'options par exemple. Pour la suite du développement de cette API et de l'agence, nous avons créé un scraper capable de récupérer des données des restaurants sur internet en python. Le script est présent dans le dossier scripts.

Ce script est la version numéro 1. Il est capable de récupérer des informations sur le restaurant, mais nous voulons aller plus loin et récupérer les données du responsable

du restaurants en allant sur différents annuaires (comme les pages jaunes par exemple) et rechercher le restaurant spécifique.